



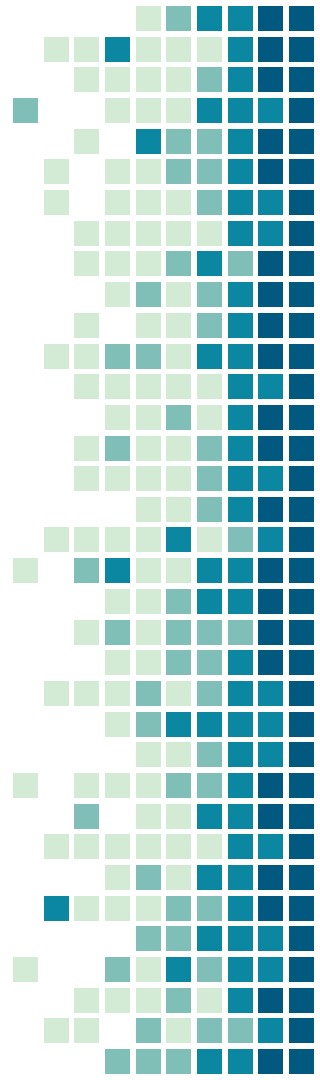
CdL in Informatica – A.A. 2024 – 2025

Programmazione 1 – Modulo 2

Lezione 9
12/11/2024

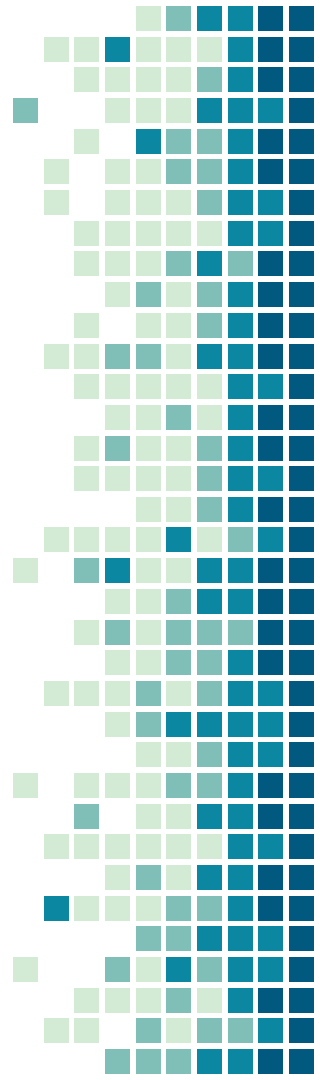
Andrea Loddo

Federico Meloni - Alessandra Perniciano - Fabio Pili



Argomenti

- Subroutine (sottoprogrammi)
 - Dichiarazione, definizione e invocazione
 - Funzioni vs. procedure
 - Passaggio di parametri
 - Subroutine e array
- Variabili static, const, globali



Subroutine

Finora abbiamo scritto programmi monolitici:

- main come entry point
- main con tutte le istruzioni (un unico blocco tuttofare)
- flusso d'esecuzione lineare e sequenziale (alterabile con selezioni e iterazioni)

In generale, meglio:

- evitare di usare un main monolitico
- suddividere il codice in moduli che risolvono sottoproblemi. Unendoli, si ottiene la soluzione del problema principale

Con i sottoprogrammi, miglioriamo:

- Modularità – evitiamo di riscrivere lo stesso codice più volte
- Manutenibilità – agiamo solo sul modulo d'interesse
- Leggibilità

```
/*Direttive di preprocessing*/  
  
int main( )  
{  
    /*tutto ciò che il programma  
    deve svolgere va qui*/  
    return 0;  
}
```

Esempio: astrazione

```
int main( )           // 1. Entry point
{
    float num, radice; // 2. Parte dichiarativa del main()
    scanf("%f", num);  // 3. scanf d'esempio
    radice = sqrt(num); /* 4. chiamata a sqrt: il main cede il
                        // controllo alla subroutine sqrt() */
                        // 5. sqrt termina, il controllo torna al main
    printf("%f", radice); // 6. chiamata alla printf e cessione del
                        // controllo da parte del main.
    return 0;          // 7. Exit point e fine del programma
}
```

- Il sottoprogramma sqrt è definito nella libreria math.
- I sottoprogrammi scanf, printf sono definiti nella libreria stdio.
- Astrazione:
 - come sono implementate le funzioni?
 - è importante saperlo?
 - cosa dobbiamo conoscere per utilizzarle?

Subroutine: dichiarazione

```
// Sintassi:  
tipo nome_sottoprogramma ( parametri )  
  
/* Esempi: */  
int somma(int, int);  
float calcolaPrezzoConIva(float prezzo, float iva);  
int print_menu( );
```

- La dichiarazione (prototipo) definisce l'intestazione della subroutine e specifica:
 - tipo del valore restituito;
 - nome del sottoprogramma;
 - tipo e nome degli eventuali parametri (bastano anche solo i tipi).
- Dove:
 - prima del main;
 - dopo le direttive di preprocessing.

Subroutine: definizione

- La definizione specifica l'implementazione della subroutine
 - ovvero: le istruzioni che la subroutine deve eseguire
- Dove:
 1. subito dopo la dichiarazione
- Osservazione sull'opzione 1:
 - ci obbliga a dichiarare seguendo un ordine preciso di utilizzo
 - se dichiariamo e definiamo s1() e poi s2(), non possiamo chiamare s2() da s1().

```
// Opzione 1: dichiarazione e definizione
int s1( int x, int y )
{
    s2( x, y ); // s2() non visibile
    return 0;
}

int s2( int x, int y )
{
    s1( x, y ); // s1() è visibile
    return 0;
}

int main( )
{
    s1( 1, 2 );
    s2( 3, 4 );

    return 0;
}
```

Subroutine: definizione

- La definizione specifica l'implementazione della subroutine
 - ovvero: le istruzioni che la subroutine deve eseguire
- Dove:
 1. subito dopo la dichiarazione
 2. dopo la graffa che chiude il main
- **Meglio usare l'opzione 2**, soprattutto in progetti di grosse dimensioni.

```
// Opzione 2
// Dichiarazione delle subroutine
int s1( int x, int y );
int s2( int x, int y );

int main( ){
    s1( 1, 2 );
    s2( 3, 4 );

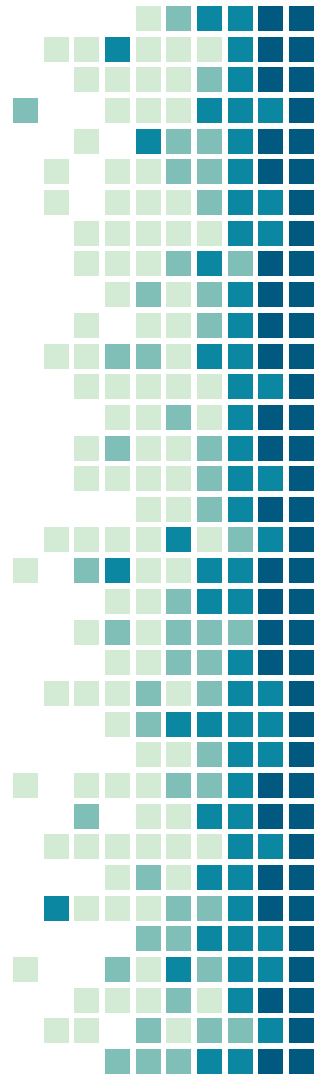
    return 0;
}

// Definizione delle subroutine
int s1( int x, int y ){
    s2( x, y ); // s2() è visibile
    return 0;
}

int s2( int x, int y ){
    s1( x, y ); // s1() è visibile
    return 0;
}
```

Definizione: precisazioni

- All'interno di una subroutine è possibile:
 - dichiarare variabili locali: saranno accessibili solo all'interno del sottoprogramma;
 - utilizzare variabili globali;
 - eseguire qualsiasi tipo di istruzione del linguaggio C;
 - invocare altri sottoprogrammi.
- All'interno di una subroutine non è possibile:
 - utilizzare variabili dichiarate altrove (nel main, in altri sottoprogrammi, ecc);
 - richiamare il main
 - o meglio: sarebbe meglio non farlo...



Subroutine: invocazione

```
int main( ) {  
    int x = 10, y = 5, sum = 0;  
    /* Invocazione del sottoprogramma somma e  
    passaggio di 2 parametri: x, y. Infine, il  
    risultato è assegnato alla variabile sum. */  
    sum = somma(x, y);  
}
```

- Dopo averla dichiarata e definita, per essere usata la subroutine va invocata (o chiamata)
- Il valore di ritorno è un'espressione del tipo restituito e può essere:
 - memorizzato in una variabile;
 - inserito in un'espressione;
 - utilizzato in un controllo.
- Le subroutine:
 - non devono necessariamente restituire un valore (esempio: una subroutine di stampa).
 - non hanno necessariamente bisogno di ricevere parametri.
- Per indicare l'assenza di parametri e/o di un valore restituito, si utilizza un tipo particolare, il tipo void.

Invocazione: esempi

```
/* Prototipo di procedura */  
void eseguiScelta( int scelta );
```

```
/* Esempio di chiamata */  
eseguiScelta( 5 );
```

```
/* Prototipo di funzione */  
int somma( int x, int y );
```

```
/* Esempio di chiamata */  
s = somma( 10, 20 );
```

```
/* Prototipo di procedura senza parametri */  
void stampaMenu( );
```

```
/* Esempio di chiamata */  
stampaMenu( );
```

```
/* Prototipo di funzione senza parametri */  
float tassoInteresse( void );
```

```
/* Esempio di chiamata */  
ti = tassoInteresse( );
```

- Procedura: sottoprogramma che non restituisce un valore.
- Funzione: sottoprogramma che restituisce un valore.

Oltre a indicare il tipo di dato, per restituire un valore c'è bisogno della keyword return.

L'istruzione return

```
// Sintassi  
return espressione;
```

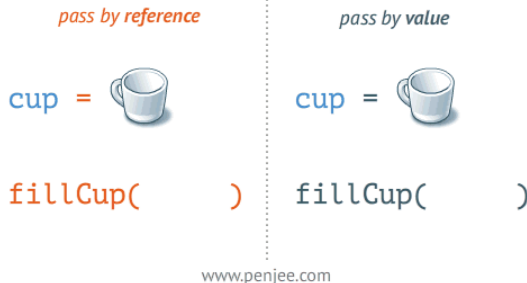
- Semantica:
 - viene valutato il valore di espressione
 - tale valore viene restituito al chiamante.
 - NB: il valore deve essere coerente con il tipo di dato specificato nella dichiarazione!
- Regole:
 - una funzione può restituire solo un valore
 - più istruzioni di return possono avere senso solo se alternative tra loro (per esempio: inserite in costrutti di selezione)

```
int subroutine( int x, int y ){  
    if(x>0) return x+y;  
    else    return x-y;  
}
```

```
int subroutine( int x, int y ){  
    int sum = x+y;  
    int dif = x-y;  
    return sum;  
    return dif; // viene ignorata  
}
```

- Esistono altre modalità per restituire più di un valore:
 - strutture;
 - puntatori.

Parametri



- La posizione con cui si passano i parametri è fondamentale.
- In generale, esistono due modalità di passaggio dei parametri:
 - per valore: i parametri che vengono passati a un sottoprogramma vengono copiati in nuove variabili che sono visibili solo al sottoprogramma.
 - per indirizzo (o riferimento): i parametri che vengono passati a un sottoprogramma sono quelli **originali**, NON copiati.
- In linguaggio C, esiste unicamente il passaggio per valore.
 - Gli array, per ora, fanno eccezione a questa regola. Vedremo il motivo quando studieremo i puntatori.
- In particolare, si definiscono i parametri:
 - attuali: quelli che il chiamante fornisce al sottoprogramma;
 - formali: quelli che il sottoprogramma riceve e utilizza.

Parametri e regole di visibilità

- In generale, vale la regola che una variabile è visibile solamente nel blocco in cui è stata dichiarata.
- Quando il controllo passa a una particolare funzione viene allocato in memoria lo spazio per le sue variabili locali e tale spazio viene rilasciato quando il controllo ritorna al chiamante.
- Esiste una variante a questa regola: le variabili globali.

```
int somma( int a, int b );

int main()
{
    int x = 5, y = 10;
    int sum;
    sum = somma(x, y); // par. attuali

    printf("La somma e' %d", sum);

    return 0;
}

int somma(int a, int b) // par. formali
{
    /* sum non e' visibile dal main() e
    cesserà di esistere al termine del
    sottoprogramma. */

    int sum = a + b;
    return sum;
}
```

Variabili globali

- Variabili globali: dichiarate all'esterno del main o delle funzioni (scope globale).
- Sono accessibili da qualsiasi punto del codice (il main e le altre funzioni) e modificabili senza bisogno di passarle come parametri.
- Sconsigliate perché:
 - difficile tenere traccia di quali funzioni ne modificano il valore (in caso di errore, dove controllare?)
 - causa di bug di non facile individuazione a causa dell'alterazione del flusso di esecuzione
 - scarso riutilizzo delle funzioni basate su variabili globali
- Meglio il passaggio di parametri.

```
void fun ( void );

/* Variabile globale: */
int numero = 0;

int main ()
{
    printf("Valore: %d", numero);
    fun();
    printf("Valore: %d", numero);
    return 0;
}

void fun ( void )
{
    scanf ( "%d" , &numero );
}
```

Passaggio di array alle subroutine

- È possibile passare un array come parametro di una subroutine in C:

```
void stampa(int[]); // prototipo subroutine

// definizione subroutine
void stampa( int array[] ){
    int i;
    for(i=0; i<DIM; i++)
        printf("%3d", array[i]);
}
```

- Si specifica che il parametro sarà un array con la coppia di parentesi quadre, eventualmente senza specificarne la dimensione (int[]).
- La dimensione può essere una define o passata come parametro
- È sempre consigliabile passare come parametro anche la dimensione: aiuta a generalizzare il codice che scriviamo (scrivere una procedura che stampa solamente DIM elementi è limitante).
- Naturalmente, è possibile, all'interno della subroutine, usare l'operatore di indicizzazione per accedere alle singole posizioni dell'array.
- NB: ogni modifica effettuata su un array è sempre visibile anche nella funzione chiamante.

Passaggio di matrici alle subroutine

- Per passare come parametri array pluridimensionali, è necessario specificare almeno le dimensioni successive alla prima:

```
#define C 5
void stampaMatrice( int matrice[][C] ); // prototipo subroutine

// definizione subroutine
void stampaMatrice( int matrice[][C] ){
    int i, j;
    for(i=0; i<R; i++) {
        for (j=0; j<C; j++)
            printf("%3d", matrice[i][j]);
        printf("\n");
    }
}
```

- Nel caso di una matrice, si specifica almeno la seconda dimensione (int[][C]).
- È sempre consigliabile passare come parametro anche le dimensioni (stessi motivi visti per gli array):

```
void stampaMatrice( int r, int c, int matrice[][c] );
```

- NB: ogni modifica effettuata su un array pluridimensionale è sempre visibile anche nella funzione chiamante.

Variabili static

- Antepoendo la keyword static alla dichiarazione di una variabile all'interno di un blocco (quale, per esempio, una funzione) se ne modifica la durata di memorizzazione e lo scope.
- Durata di memorizzazione: una variabile static non viene distrutta all'uscita dal blocco, ma rimane in memoria, mantenendo staticamente il suo valore. Inoltre, viene inizializzata una sola volta.
- Scope: una variabile static può essere vista e modificata soltanto nello scope in cui è stata dichiarata.

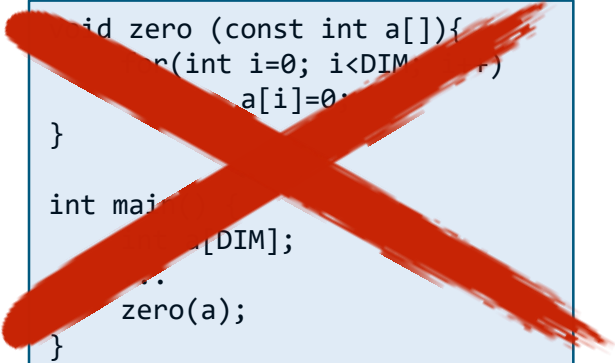
```
void test ( void ) {  
    static int numero = 0;  
    numero++;  
    printf("%d\n", numero);  
}  
int main () {  
    test(); // Stampa 1  
    test(); // Stampa 2  
    test(); // Stampa 3  
  
    return 0;  
}
```

- NB: la stampa senza static sarebbe stata 1, 1, 1.

Variabili const

- Antepoendo la keyword `const` alla dichiarazione ed eventuale inizializzazione di una variabile, o nella dichiarazione dei parametri di una funzione, la si trasforma in un oggetto di sola lettura.
- Diversa dalla `#define`:
 - La `define` consente di definire macro (numeri, caratteri o stringhe), la keyword `const` può essere anteposta a qualsiasi tipo di oggetto (array, strutture, ecc...)
 - Le `const` sono soggette alle regole di scope delle variabili, mentre le macro no
 - Le macro sono gestite dal preprocessore, e non dal compilatore.

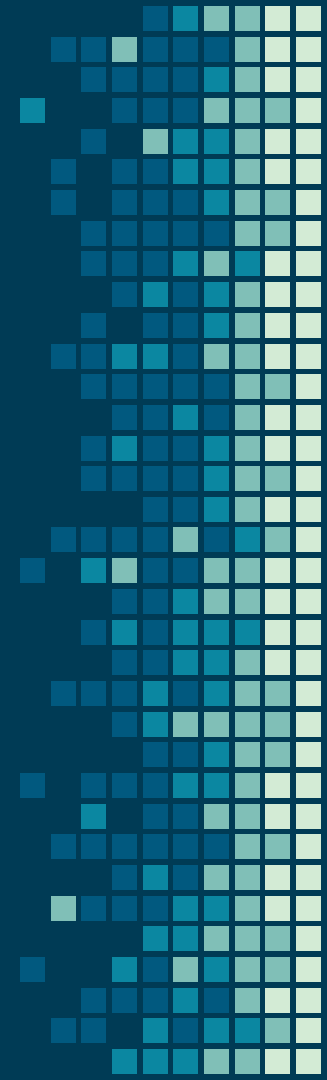
```
void stampa (const int a[]){  
    for(i=0; i<DIM; i++)  
        printf("%d", a[i]);  
}  
  
int main(){  
    int a[DIM];  
    ...  
    stampa(a);  
}
```



```
void zero (const int a[]){  
    for(int i=0; i<DIM; i++)  
        a[i]=0;  
}  
  
int main(){  
    int a[DIM];  
    ...  
    zero(a);  
}
```

FINE!

Domande?



Autovalutazione

1. Che differenza c'è tra funzioni e procedure?
2. È possibile utilizzare una subroutine prima di averla dichiarata?
3. Il tipo della funzione è obbligatorio?
4. Che differenza c'è tra dichiarazione e definizione di una subroutine?
5. È possibile invocare una funzione senza passare alcun parametro?
6. Quali costrutti e istruzioni si possono usare all'interno di una subroutine?
7. È possibile invocare il main da una funzione? Nel caso sia possibile, è consigliato farlo?
8. A cosa serve l'istruzione return ?
9. In una procedura è obbligatorio mettere l'istruzione return ?
10. È possibile passare un array come parametro? Se sì, qual è la caratteristica di questo passaggio?
11. Cos'è una variabile globale?
12. È possibile modificare il valore di una variabile globale dentro una procedura anche se inizializzata nel main?
13. Cos'è una variabile static?
14. Cos'è una variabile const?
15. Posso dichiarare un array di dimensione n, con n dichiarato come const int?

Esercizi

1. Scrivere la funzione `randRange` che riceva in ingresso due numeri interi (min e max), acquisiti nel main, e restituisca un valore generato casualmente all'interno del range i cui gli estremi sono i due valori passati in ingresso (estremi inclusi). Invocare tale funzione nel main passando i valori necessari, poi stampare il valore restituito dalla funzione.
NB: l'inizializzazione del seed va fatta nel main.
VARIANTE: scrivere una funzione `randRangeFloat` per generare numeri con la virgola. La funzione riceverà un ulteriore parametro che rappresenta il numero di cifre decimali richiesto.
2. Scrivere un programma che acquisisca un numero e stampi un conto alla rovescia a partire dal numero acquisito. Realizzare una procedura che si occupi di stampare il conto alla rovescia.
3. Scrivere una funzione che esegua l'acquisizione da input di un numero e il controllo che il numero si trovi all'interno di un determinato intervallo (min, max), mediante l'acquisizione ripetuta del numero. La funzione deve restituire sempre un valore compreso tra min e max. Stampare tale numero nel main.
4. Scrivere una funzione `calcolaGiorno(giorno, mese, anno)` che restituisca il giorno dell'anno (un numero compreso tra 1 e 366) specificato dai tre argomenti. La funzione deve verificare se l'anno è bisestile o meno, ed effettuare il calcolo di conseguenza. Nel main, impostare una data e stampare in output il giorno dell'anno.
 - NB: un anno è bisestile se è divisibile per 400; altrimenti se è divisibile per 4 ma non per 100.
 - Modificare, poi, l'esercizio per rendere il calcolo dell'anno bisestile una funzione.

Esercizi

5. Scrivere una funzione che faccia il clamp tra 50 e 100 di un numero passato come parametro.
ES: x: 7 -> f: 50; x: 75 -> f: 75; x: 135 -> f: 100
6. Scrivere una funzione che prenda come parametro un carattere e:
 - se è una lettera minuscola, restituisce una lettera maiuscola;
 - se è una lettera maiuscola, restituisce una lettera minuscola;
 - lascia invariati tutti gli altri caratteri.
 - Modificare successivamente una stringa dichiarata nel main modificando ogni carattere utilizzando la funzione appena descritta.
7. Scrivere due procedure che andranno richiamate nel main:
 - stampaArray che prenda come parametro un array di interi e ne stampi il suo contenuto.
 - generaArray che prenda come parametro un array di interi e ne popoli il suo contenuto mediante randRange dell'esercizio 1.
8. Scrivere due procedure che andranno richiamate nel main:
 - popolaMatrice che riceva come parametro una matrice di float e generi casualmente i suoi elementi in un range definito dall'utente, sfruttando randRangeFloat dell'esercizio 1; anche righe e colonne vanno generate casualmente.
 - stampaMatrice che prenda come parametro una matrice di float e ne stampi il suo contenuto.